

JavaScript Notes

Strings · Arrays · Objects · Nested Objects · Math

A beginner-friendly, revision-ready reference

📖 Table of Contents

1. 📖 [Strings](#)
2. 📖 [Method Chaining](#)
3. 📖 [Arrays](#)
4. 📖 [Objects](#)
5. 📖 [Nested Objects](#)
6. 📖 [Math Object](#)
7. 📖 [Random Numbers](#)
8. 📖 [Important Comparisons](#)
9. 📖 [Common Mistakes](#)
10. 📖 [Quick Revision Cheat Sheet](#)
11. 📖 [Interview / Viva Questions](#)
12. 📖 [Practice Problems](#)
13. 📖 [Final Takeaways](#)

1. ? Strings

A **string** is a sequence of characters, used to represent text.

```
let msg = "Hello World";
```

? Strings are **immutable** in JavaScript. Every string method that appears to "modify" a string is actually returning a **brand-new string**, leaving the original untouched.

trim()

? What is it?

Removes whitespace from the beginning and end of a string only.

? Syntax

```
string.trim()
```

? Example

```
let msg = "    Hello World    ";  
console.log(msg.trim());
```

Output

```
Hello World
```

? How it works

trim() scans from both ends of the string and strips whitespace until it hits actual content. It does not touch spaces in the middle:

? Does not modify the original string — it returns a new one. Does not remove spaces between words. Related: trimStart() and trimEnd() trim only one side.

Example of middle spaces being preserved:

```
let msg = "    Hello    World    ";  
console.log(msg.trim());
```

Output

```
Hello    World
```

slice()

What is it?

Extracts a portion of a string and returns it as a new string, without changing the original.

Syntax

```
string.slice(start, end)
```

Example

```
let msg = "Hello World Coding";
console.log(msg.slice(6));
```

Output

```
World Coding
```

How it works

Every character has an index starting at 0. slice(6) means: start at index 6 and take everything until the end. When a second argument is given, that end index is excluded.

Negative indexes count backward from the end of the string — slice(-5) grabs the last 5 characters.

slice() never modifies the original string; it always returns a new one.

H	e	l	l	o		W	o	r	l	d		C	o	d	i	n	g
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17

replace()

What is it?

Replaces the first matching occurrence of a substring with a new value.

Syntax

```
string.replace(oldValue, newValue)
```

Example

```
let msg = "Hello World World";
console.log(msg.replace("World", "JavaScript"));
```

Output

```
Hello JavaScript World
```

📖 How it works

`replace()` scans left to right, finds the first match, swaps it out, and stops — later matches are left alone.

📖 Returns a new string; the original is never changed.

replaceAll()

📖 What is it?

Replaces every matching occurrence of a substring.

📖 Syntax

```
string.replaceAll(oldValue, newValue)
```

📖 Example

```
let msg = "Hello World World";  
console.log(msg.replaceAll("World", "JavaScript"));
```

Output

```
Hello JavaScript JavaScript
```

📖 How it works

Unlike `replace()`, this keeps scanning the entire string and swaps out every match it finds.

📖 Also returns a new string — the original remains unchanged.

repeat()

📖 What is it?

Repeats a string a specified number of times and returns the result as a new string.

📖 Syntax

```
string.repeat(count)
```

📖 Example

```
let msg = "Mango";  
console.log(msg.repeat(5));
```

Output

```
MangoMangoMangoMangoMango
```

🔗 How it works

`repeat()` simply glues the string to itself count times, in order — e.g. `"Hi ".repeat(3)` gives `"Hi Hi Hi "`.

🔗 Does not modify the original string.

2. 🔗 Method Chaining

🔗 What is it?

Method chaining means calling one method directly on the result of another — one after the other, in a single line.

```
let msg = "    apple    ";
console.log(msg.toUpperCase().trim());
```

🔗 Visual flow

```
msg
↓
toUpperCase()
↓
"    APPLE    "
↓
trim()
↓
"APPLE"
```

This is effectively identical to writing it in two separate steps:

```
let result1 = msg.toUpperCase();
let result2 = result1.trim();

console.log(result2); // "APPLE"
```

🔗 Chaining only works if the method before it returns a value the next method can act on. `toUpperCase()` returns a string, so `trim()` — a string method — can follow it directly.

3. 📖 Arrays

An **array** is an ordered list of values. This section focuses on two of the most important — and most misunderstood — array methods: `splice()` and `sort()`.

splice()

📖 What is it?

`splice()` is an **array** method (not a string method) that can remove, add, or replace elements — all while modifying the array directly.

Remove

```
array.splice(start, deleteCount)

let arr = ["mango", "apple", "banana", "grapes", "dragonfruit"];
console.log(arr.splice(0, 3));
```

Output

```
["mango", "apple", "banana"]
```

`splice(0, 3)` means: start at index 0, delete 3 elements. `splice()` also **returns** the removed elements. After this call, `arr` becomes:

Output

```
["grapes", "dragonfruit"]
```

Add

```
array.splice(start, 0, item)

let arr = ["grapes", "dragonfruit"];
arr.splice(1, 0, "orange", "pineapple");

console.log(arr);
```

Output

```
["grapes", "orange", "pineapple", "dragonfruit"]
```

The 0 for `deleteCount` means "delete nothing" — so the new items are simply inserted at index 1.

Replace

```
array.splice(start, deleteCount, item)

let arr = ["apple", "banana", "mango"];
arr.splice(1, 1, "orange");

console.log(arr);
```

Output

```
["apple", "orange", "mango"]
```

🔗 Unlike most string methods, `splice()` modifies the original array directly.

```
let arr = [1, 2, 3];  
arr.splice(1, 1);  
  
console.log(arr); // [1, 3]
```

sort()

🔗 What is it?

Sorts the elements of an array **in place**.

🔗 Syntax

```
array.sort()
```

🔗 Example

```
let arr = [5, 3, 1, 4, 2];  
arr.sort();  
  
console.log(arr);
```

Output

```
[1, 2, 3, 4, 5]
```

That looks correct — but watch what happens with these numbers:

```
let arr = [10, 2, 5, 1];  
console.log(arr.sort());
```

Output

```
[1, 10, 2, 5]
```

🔗 By default, `sort()` converts every element to a **string** and compares them lexicographically, not numerically — that's why 10 sorts before 2.

Ascending (numeric)

```
array.sort((a, b) => a - b);
```



```
let arr = [10, 2, 5, 1];  
arr.sort((a, b) => a - b);  
  
console.log(arr);
```

Output

```
[1, 2, 5, 10]
```

Descending (numeric)

```
array.sort((a, b) => b - a);
```

Output

```
[10, 5, 2, 1]
```

🔗 How the comparator works

- If $a - b < 0 \rightarrow a$ is placed before b
- If $a - b > 0 \rightarrow b$ is placed before a
- If $a - b === 0 \rightarrow$ their relative order stays the same

🔗 `sort()` modifies the original array and returns a reference to that same, now-sorted array.

4. 📖 Objects

📖 What is it?

An **object** stores data as a collection of key-value pairs.

```
const student = {
  name: "John",
  age: 20,
  grade: "A",
  isEnrolled: true
};
```

📖 Visual structure

```
student
├── --- name      → "John"
├── --- age       → 20
├── --- grade     → "A"
└── --- isEnrolled → true
```

Accessing Properties

Dot notation

```
console.log(student.name); // "John"
console.log(student.age);  // 20
```

Bracket notation

```
console.log(student["name"]); // "John"
```

Dot Notation vs Bracket Notation

Notation	Best used when...
<code>student.name</code>	The property name is known directly, written literally in the code
<code>student["name"]</code>	The property name is stored in a variable, or contains special characters/spaces

```
let property = "name";
console.log(student[property]); // "John"
```

❗ `student[property]` looks up whatever value `property` holds ("name"). This is completely different from `student.property`, which literally searches for a key called "property".

5. ? Nested Objects

? What is it?

A **nested object** is simply an object stored as the value of a key inside another object.

```
const classInfo = {  
  student1: {  
    name: "Adrija",  
    age: 20,  
    grade: "A"  
  },  
  
  student2: {  
    name: "Diganta",  
    age: 21,  
    grade: "A+"  
  },  
  
  student3: {  
    name: "Saif",  
    age: 22,  
    grade: "B+"  
  }  
};
```

? Visual structure

```
classInfo  
├── student1  
│   ├── name  
│   ├── age  
│   └── grade  
├── student2  
│   ├── name  
│   ├── age  
│   └── grade  
└── student3  
    ├── name  
    ├── age  
    └── grade
```

Accessing Nested Objects

```
console.log(classInfo.student1.name);
```

Output

Adrija

How it works

```
classInfo
  ↓
student1
  ↓
name
  ↓
"Adrija"
```

```
console.log(classInfo.student2.age); // 21
```

6. Math Object

The built-in **Math** object provides ready-to-use mathematical operations. It is not a constructor — you never create it, you simply call its methods directly.

```
Math.method()
```

Math.abs()

What

Returns the absolute (non-negative) value of a number.

Example

```
console.log(Math.abs(-5));
```

Output

```
5
```

How it works

Think of it as removing the negative sign, if there is one: $\text{Math.abs}(-10) \rightarrow 10$, $\text{Math.abs}(10) \rightarrow 10$, $\text{Math.abs}(0) \rightarrow 0$.

Math.max()

What

Returns the largest of the given numbers.

Example

```
console.log(Math.max(5, 10, 15, 20));
```

Output

```
20
```

Math.min()

What

Returns the smallest of the given numbers.

Example

```
console.log(Math.min(5, 10, 15, 20));
```

Output

```
5
```

Math.sqrt()

What

Returns the square root of a number.

Example

```
console.log(Math.sqrt(25));
```

Output

```
5
```

How it works

Also: $\text{Math.sqrt}(16) \rightarrow 4$, $\text{Math.sqrt}(81) \rightarrow 9$.

Math.pow()

What

Raises a number to a given power.

Example

```
console.log(Math.pow(2, 3));
```

Output

```
8
```

How it works

$2^3 = 2 \times 2 \times 2 = 8$. Modern JavaScript also supports the exponent operator $2 ** 3$, which gives the same result.

Math.round()

What

Rounds to the nearest integer.

Example

```
Math.round(2.4); // 2  
Math.round(2.5); // 3  
Math.round(2.7); // 3
```

How it works

Anything .5 or above rounds up; anything below .5 rounds down.

Math.ceil()

What

Rounds up toward positive infinity ("ceiling").

Example

```
console.log(Math.ceil(2.3));
```

Output

```
3
```

How it works

Also: $\text{Math.ceil}(2.9) \rightarrow 3$, $\text{Math.ceil}(5.1) \rightarrow 6$.

Math.floor()

What

Rounds down toward negative infinity ("floor").

Example

```
console.log(Math.floor(2.7));
```

Output

```
2
```

How it works

Also: $\text{Math.floor}(2.9) \rightarrow 2$, $\text{Math.floor}(5.1) \rightarrow 5$.

For 2.7 → round gives 3, ceil gives 3, floor gives 2.

7. ? Random Numbers

Math.random()

? What

Generates a pseudo-random floating-point number.

```
console.log(Math.random());
```

Possible outputs: 0.34827, 0.91231, 0.00451, etc.

? The exact range is $0 \leq \text{Math.random()} < 1$. It can produce 0, but it will never produce 1.

Generating a Random Integer in a Range

Rather than memorizing the formula, here's how to derive it step by step, using the example of a random number from **1 to 10**.

Step 1 — Start with Math.random():

Output

```
0 <= x < 1
```

Step 2 — Multiply by 10 to widen the range:

```
Math.random() * 10
```

Output

```
0 <= x < 10
```

Step 3 — Apply Math.floor() to make it a whole number:

```
Math.floor(Math.random() * 10)
```

Output

```
Possible values: 0, 1, 2, 3, 4, 5, 6, 7, 8, 9
```

Step 4 — Add 1 to shift the range to start at 1:

```
Math.floor(Math.random() * 10) + 1
```

Output

```
Possible values: 1, 2, 3, 4, 5, 6, 7, 8, 9, 10
```

The general formula

```
Math.floor(Math.random() * (max - min + 1)) + min
```

Examples:

```
// Random number from 1-10  
Math.floor(Math.random() * 10) + 1;  
  
// Random number from 1-100  
Math.floor(Math.random() * 100) + 1;  
  
// Random number from 20-50  
Math.floor(Math.random() * 31) + 20;
```

❓ Why $\text{max} - \text{min} + 1$? That's the count of possible integers in the range (inclusive of both ends). Multiplying `Math.random()` by that count and flooring gives an offset from 0 up to $\text{max} - \text{min}$; adding `min` shifts it into the correct range.

8. ? Important Comparisons

String Methods

Method	Purpose	Modifies Original?
<code>trim()</code>	Removes whitespace from both ends	✗
<code>slice()</code>	Extracts part of a string	✗
<code>replace()</code>	Replaces the first matching occurrence	✗
<code>replaceAll()</code>	Replaces all matching occurrences	✗
<code>repeat()</code>	Repeats a string n times	✗

Array Methods

Method	Purpose	Modifies Original?
<code>splice()</code>	Adds, removes, or replaces elements	✓
<code>sort()</code>	Sorts the array's elements	✓

9. ❓ Common Mistakes

Mistake 1 – Confusing `slice()` and `splice()`

❗ `slice()` (string or array) returns a new copy without touching the original. `splice()` (array only) modifies the original array directly.

Mistake 2 – Thinking `replace()` replaces everything

❗ `replace()` only swaps the first match. Use `replaceAll()` to replace every match.

Mistake 3 – Thinking `trim()` removes spaces everywhere

❗ `trim()` only removes whitespace from the start and end of a string — spaces between words are untouched.

Mistake 4 – Forgetting `sort()`'s default behavior

❗ By default, `sort()` compares elements as strings, which is why `[10, 2, 5, 1].sort()` doesn't come out numerically ordered. Always pass a comparator for numeric sorting.

Mistake 5 – Forgetting that `splice()` changes the original array

❗ `splice()` mutates the array it's called on — there's no separate "unmodified" copy left behind.

Mistake 6 – Confusing dot and bracket notation

❗ `object.property` looks for a key literally named "property". `object[property]` looks up whatever value the variable `property` currently holds.

Mistake 7 – Misunderstanding `Math.random()`'s range

❗ `Math.random()` returns a value from 0 (inclusive) up to, but never including, 1.

10. 📄 Quick Revision Cheat Sheet

Method	Used With	Purpose	Mutates?
<code>trim()</code>	String	Removes whitespace from both ends	✗
<code>slice()</code>	String	Extracts a portion of a string	✗
<code>replace()</code>	String	Replaces first matching occurrence	✗
<code>replaceAll()</code>	String	Replaces all matching occurrences	✗
<code>repeat()</code>	String	Repeats a string n times	✗
<code>splice()</code>	Array	Adds/removes/replaces elements	✓
<code>sort()</code>	Array	Sorts array elements	✓
<code>sort((a,b)=>a-b)</code>	Array	Numeric ascending sort	✓
<code>sort((a,b)=>b-a)</code>	Array	Numeric descending sort	✓
<code>Math.abs()</code>	Math	Absolute value	—
<code>Math.max()</code>	Math	Largest value	—
<code>Math.min()</code>	Math	Smallest value	—
<code>Math.sqrt()</code>	Math	Square root	—
<code>Math.pow()</code>	Math	Raises to a power	—
<code>Math.round()</code>	Math	Rounds to nearest integer	—
<code>Math.ceil()</code>	Math	Rounds up	—
<code>Math.floor()</code>	Math	Rounds down	—
<code>Math.random()</code>	Math	Random float $0 \leq x < 1$	—

📄 Category Cheat Sheet

STRING ↓ <code>trim()</code> <code>slice()</code> <code>replace()</code> <code>replaceAll()</code> <code>repeat()</code>	ARRAY ↓ <code>splice()</code> <code>sort()</code>	MATH ↓ <code>abs()</code> <code>max()</code> <code>min()</code> <code>sqrt()</code> <code>pow()</code> <code>round()</code> <code>ceil()</code> <code>floor()</code> <code>random()</code>	OBJECT ↓ key : value ↓ <code>student.name</code> <code>student.age</code> <code>student.grade</code>
--	--	--	--

11. ? Interview / Viva Questions

Q1. What is the difference between `slice()` and `splice()`?

A: `slice()` (string or array) returns a new copy and leaves the original untouched. `splice()` (array only) modifies the original array directly.

Q2. Does `trim()` modify the original string?

A: No — it returns a new string with whitespace removed from both ends; the original is unchanged.

Q3. What is method chaining?

A: Calling one method directly on the return value of another, e.g. `msg.toUpperCase().trim()`.

Q4. What is the difference between `replace()` and `replaceAll()`?

A: `replace()` swaps only the first matching occurrence; `replaceAll()` swaps every matching occurrence.

Q5. Why does `numeric sort()` sometimes behave unexpectedly?

A: By default `sort()` compares elements as strings, not numbers, producing lexicographical rather than numeric order.

Q6. What does `(a, b) => a - b` do inside `sort()`?

A: It's a comparator: negative result → a comes first; positive → b comes first; zero → order unchanged. This produces ascending numeric order.

Q7. What is an object?

A: A data structure that stores information as key-value pairs.

Q8. What is a nested object?

A: An object whose value for one or more of its keys is itself another object.

Q9. What's the difference between dot and bracket notation?

A: Dot notation (`obj.key`) is used when the property name is known and written literally. Bracket notation (`obj["key"]` or `obj[variable]`) is used when the property name is dynamic.

Q10. What is `Math.random()`'s range?

A: From 0 (inclusive) up to, but not including, 1.

Q11. How can you generate a random integer between two numbers?

A: `Math.floor(Math.random() * (max - min + 1)) + min`

Q12. What's the difference between `Math.ceil()` and `Math.floor()`?

A: `ceil()` always rounds up to the next integer; `floor()` always rounds down to the previous integer.

Q13. Are strings mutable or immutable in JavaScript?

A: Immutable — every "modifying" method actually returns a brand-new string.

Q14. What does `splice(start, 0, item)` do?

A: Since `deleteCount` is 0, nothing is removed — `item` is simply inserted at the start index.

Q15. Difference between `Math.round(2.3)` and `Math.ceil(2.3)`?

A: `Math.round(2.3)` gives 2 (rounds to nearest), while `Math.ceil(2.3)` gives 3 (always rounds up).

12. 📖 Practice Problems

📖 No solutions provided — try each one yourself!

Challenge 1

```
let msg = "    JavaScript    ";
```

Remove the unnecessary whitespace and convert the result to uppercase.

Challenge 2

Given let text = "I like cats. Cats are great. I want more cats.", replace only the first occurrence of "cats" with "dogs".

Challenge 3

Using the same string as above, replace every occurrence of "cats" with "dogs".

Challenge 4

```
let str = "Programming";
```

Use slice() to extract just "gram" from it.

Challenge 5

Write a line of code that prints the word "Ha" repeated 4 times in a row using repeat().

Challenge 6

```
let arr = ["a", "b", "c", "d", "e"];
```

Use splice() to remove "c" and "d" only.

Challenge 7

```
let arr = ["a", "e"];
```

Use splice() to insert "b", "c", and "d" between them so the array reads ["a","b","c","d","e"].

Challenge 8

```
let nums = [40, 100, 1, 5, 25];
```

Sort the array numerically in ascending order.

Challenge 9

Sort the same array in descending order.

Challenge 10

Create an object called book with keys title, author, and year. Access title using dot notation and author using bracket notation.

Challenge 11

Create a nested object company with two departments (sales, engineering), each with head and teamSize. Log the head of the engineering department.

Challenge 12

Write an expression that generates a random integer between 50 and 75 (inclusive) using Math.random() and Math.floor().

13. 📖 Final Takeaways

- Strings are **immutable** — `trim()`, `slice()`, `replace()`, and `repeat()` always return a new string.
- `slice()` extracts a substring; its end index is excluded, and negative indexes count from the end.
- `replace()` only changes the first match; `replaceAll()` changes every match.
- Method chaining runs left to right, each method acting on the result of the one before it.
- `splice()` is an array method that removes, adds, or replaces elements — and modifies the original array.
- `sort()` also modifies the original array.
- By default, `sort()` compares elements as strings, which can produce incorrect results for numbers.
- Use `(a, b) => a - b` for ascending numeric sort, and `(a, b) => b - a` for descending.
- Objects store data as key-value pairs, accessible via dot notation or bracket notation.
- Bracket notation is required when the property name is stored in a variable.
- Nested objects are objects stored inside other objects, accessed by chaining dot notation.
- `Math.round()`, `Math.ceil()`, and `Math.floor()` round differently — nearest, always up, and always down.
- `Math.random()` returns a float where $0 \leq x < 1$, never reaching 1.
- `Math.floor(Math.random() * (max - min + 1)) + min` generates a random integer between min and max, inclusive.
- These fundamentals set up topics like arrays of objects, destructuring, spread/rest, and `map()/filter()/reduce()/forEach()`.